

EDA Notebook

0. Setup Environment

```
In [ ]: # DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

```
===== 12.9/12.9 MB 38.3 MB/s eta 0:00:00
===== 4.9/4.9 MB 25.2 MB/s eta 0:00:00
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
Mounted at /content/gdrive

You can now save your data files in: /content/gdrive/MyDrive/36106/assignment/AT3/data

Student Information

```
In [ ]: # <Student to fill this section>
group_name = "36106-26AU-AT3-Group 24"
student_name = "Nana Ama Goldwater"
student_id = "26137455"
```

```
In [ ]: # Do not modify this code
print_tile(size="h1", key='group_name', value=group_name)
```

group_name

36106-26AU-AT3-Group 24

```
In [ ]: # Do not modify this code
print_tile(size="h1", key='student_name', value=student_name)
```

student_name

Nana Ama Goldwater

```
In [ ]: # Do not modify this code
print_tile(size="h1", key='student_id', value=student_id)
```

student_id

26137455

0. Python Packages

0.a Install Additional Packages

If you are using additional packages, you need to install them here using the command: !
`pip install <package_name>`

```
In [ ]: # <Student to fill this section>
```

0.b Import Packages

```
In [ ]: import pandas as pd
import altair as alt
import numpy as np

alt.data_transformers.disable_max_rows() # >5000 rows allowed in Altair
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 200)
```

B. Data Understanding

```
In [ ]: # Do not modify this code
try:
    df = pd.read_csv(at.folder_path / "sales_order_header.csv")
except Exception as e:
    print(e)
```

[Errno 2] No such file or directory: '/content/gdrive/MyDrive/36106/assignment/AT3/data/sales_order_header.csv'

B.1 Explore Dataset

```
In [ ]: # Structural overview.
df = pd.read_csv('/content/sales_order_header.csv')
print(f'Dataset shape : {df.shape[0]:,} rows x {df.shape[1]} columns\n')
df.info()

print('\nFirst 3 rows:')
display(df.head(3))
```

Dataset shape : 31,465 rows x 17 columns

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 31465 entries, 0 to 31464
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   sales_order_id         31465 non-null  object
1   revision_number        31465 non-null  float64
2   status                 31465 non-null  float64
3   online_order_flag      31465 non-null  float64
4   customer_id            31465 non-null  object
5   sales_person_id        3806 non-null   object
6   territory_id           31465 non-null  object
7   currency_rate_id       13976 non-null  float64
8   order_date             31465 non-null  object
9   due_date               31465 non-null  object
10  ship_date              31465 non-null  object
11  sales_order_number     31465 non-null  object
12  account_number         31465 non-null  object
13  sub_total              31465 non-null  float64
14  tax_amount             31465 non-null  float64
15  freight                31465 non-null  float64
16  total_due              31465 non-null  float64
dtypes: float64(8), object(9)
memory usage: 4.1+ MB
```

First 3 rows:

	sales_order_id	revision_number	status	online_order_flag	customer_id	sales_person_id	territo
0	1b9285b8-4501-4c92- adf7- ee74b4fa3d62	8.0	5.0	0.0	038a7b5e- 5d15-4a1a- 8305- e18131e402cc	28141e19-49e8- 4090-9196- 6e8f43b10de8	1e17 7db9- e6fb9b3c
1	b28eec58-f64d- 4211-b604- bc9e801fb523	8.0	5.0	0.0	75c8fd92- 2eb9-4223- b92c- f7c66751ed52	28141e19-49e8- 4090-9196- 6e8f43b10de8	1e17 7db9- e6fb9b3c
2	664be852- 4095-4495- 887a- 4fc24bb299e6	8.0	5.0	0.0	e5107fe5- b23a-46a4- 8e49- bd195ef3b713	27d62980-8d18- 4170-95a5- 7f7bb9cada5a	c8b9 8c4c- 6954c90

```
In [ ]: # Missing values, duplicates, and uniqueness.
miss = df.isna().sum().to_frame('n_missing')
miss['pct_missing'] = (df.isna().mean() * 100).round(2)
print('Missing values per column:')
display(miss.sort_values('n_missing', ascending=False))

print(f'\nExact duplicate rows      : {df.duplicated().sum()}')
print(f'Unique sales_order_id values : {df["sales_order_id"].nunique():,} / {len(df):,}')
print(f'Unique customer_id values    : {df["customer_id"].nunique():,}')
print(f'Average orders per customer   : {len(df) / df["customer_id"].nunique():.2f}')
```

Missing values per column:

	n_missing	pct_missing
sales_person_id	27659	87.90
currency_rate_id	17489	55.58
sales_order_id	0	0.00
revision_number	0	0.00
status	0	0.00
customer_id	0	0.00
online_order_flag	0	0.00
territory_id	0	0.00
order_date	0	0.00
due_date	0	0.00
ship_date	0	0.00
sales_order_number	0	0.00
account_number	0	0.00
sub_total	0	0.00
tax_amount	0	0.00
freight	0	0.00
total_due	0	0.00

```
Exact duplicate rows      : 0
Unique sales_order_id values : 31,465 / 31,465
Unique customer_id values  : 19,119
Average orders per customer : 1.65
```

```
In [ ]: dataset_insights = """
Dataset: `sales_order_header.csv`: 31,465 rows x 17 columns. This is the
sales fact table at the order-header level (one row per order; line items
live in sales_order_detail).

Schema:
- Identifiers      : sales_order_id (PK, UUID), sales_order_number (human-readable),
```

```
customer_id, sales_person_id, territory_id, account_number,  
currency_rate_id.  
- Dates (strings) : order_date, due_date, ship_date.  
- Flags / status : revision_number, status, online_order_flag.  
- Monetary (USD) : sub_total, tax_amount, freight, total_due.
```

Quality / completeness:

- sales_order_id is unique and fully populated (31,465 / 31,465).
- No exact duplicate rows.
- All date columns load as strings and must be converted to datetime before any temporal computation (handled in prep notebook B.1).
- sales_person_id missing on 27,659 rows (87.9%), populated only when a sales representative is assigned. This corresponds 1:1 with the offline channel (see B.4): online orders have no sales rep, offline orders do. The two columns therefore carry the same information; the prep notebook drops sales_person_id and keeps online_order_flag.
- currency_rate_id missing on 17,489 rows (55.6%), populated only for non-USD transactions. Not used by the repeat-purchase model.

Granularity and repeat-purchase signal:

- 31,465 orders from 19,119 distinct customer_id values → average 1.65 orders per customer. This is a SPARSE repeat signal: more than half of customers appear only once, meaning the positive class rate for the repeat-purchase label will be moderate-to-low. The baseline notebook's emphasis on F1 / PR-AUC over accuracy is well justified.
- The 19,119 distinct customer_ids in this fact table EXCEEDS the 10,331 unique customer_ids in the customer dimension. This indicates either orphan orders (customer_ids not in the dimension) or that the customer.csv file was deduplicated more aggressively than the fact table expects. The prep notebook's left-merge handles either case safely.

Implications for the use case:

- This is the table that supplies the LABEL (orders within the prediction window) and the bulk of the FEATURES (RFM, recent activity, online ratio). Sections B.2, B.3, B.4 below explore the three columns that drive those features.

```
"""
```

```
In [ ]: # Do not modify this code  
print_tile(size="h3", key='dataset_insights', value=dataset_insights)
```

Dataset: `sales\_order\_header.csv`: 31,465 rows x 17 columns. This is the sales fact table at the order-header level (one row per order; line items live in sales_order_detail). Schema: - Identifiers : sales_order_id (PK, UUID), sales_order_number (human-readable), customer_id, sales_person_id, territory_id, account_number, currency_rate_id. - Dates (strings) : order_date, due_date, ship_date. - Flags / status : revision_number, status, online_order_flag. - Monetary (USD) : sub_total, tax_amount, freight, total_due. Quality / completeness: - sales_order_id is unique and fully populated (31,465 / 31,465). - No exact duplicate rows. - All date columns load as strings and must be converted to datetime before any temporal computation (handled in prep notebook B.1). - sales_person_id missing on 27,659 rows (87.9%), populated only when a sales representative is assigned. This corresponds 1:1 with the offline channel (see B.4): online orders have no sales rep, offline orders do. The two columns therefore carry the same information; the prep notebook drops sales_person_id and keeps online_order_flag. - currency_rate_id missing on 17,489 rows (55.6%), populated only for non-USD transactions. Not used by the repeat-purchase model. Granularity and repeat-purchase signal: - 31,465 orders from 19,119 distinct customer_id values → average 1.65 orders per customer. This is a SPARSE repeat signal: more than half of customers appear only once, meaning the positive class rate for the repeat-purchase label will be moderate-to-low. The baseline notebook's emphasis on F1 / PR-AUC over accuracy is well justified. - The 19,119 distinct customer ids in this fact table EXCEEDS the 10,331 unique customer ids in the customer dimension. This indicates either orphan orders (customer ids not in the dimension) or that the customer.csv file was deduplicated more aggressively than the fact table expects. The prep notebook's left-merge handles either case safely. Implications for the use case: - This is the table that supplies the LABEL (orders within the prediction window) and the bulk of the FEATURES (RFM, recent activity, online ratio). Sections B.2, B.3, B.4 below explore the three columns that drive those features.

B.2 Explore Feature of Interest `order_date`

```
In [ ]: # order_date drives EVERYTHING in the repeat-purchase prep pipeline (cutoff,
# recency, frequency, tenure). Convert and inspect.
od = pd.to_datetime(df['order_date'], errors='coerce')
bad = od.isna().sum()
```

```
print(f'Rows in df           : {len(df):,}')
print(f'Unparseable order_dates : {bad}')
print(f'Date range           : {od.min()} → {od.max()}')
print(f'Span                 : {(od.max() - od.min()).days:,} days')
print(f'Distinct dates        : {od.dt.normalize().nunique():,}')
```

```
Rows in df           : 31,465
Unparseable order_dates : 0
Date range           : 2011-05-30 22:00:00 → 2014-06-29 22:00:00
Span                 : 1,126 days
Distinct dates        : 1,124
```

```
In [ ]: # Visualise order volume over time (monthly bins). The trend, seasonality,
# and any abrupt change are all relevant when choosing a cutoff date.
monthly = (od.dt.to_period('M').dt.to_timestamp()
            .value_counts().sort_index()
            .rename('orders').reset_index())
monthly.columns = ['month', 'orders']

alt.Chart(monthly).mark_line(point=True).encode(
    x=alt.X('month:T', title='Order month'),
    y=alt.Y('orders:Q', title='Number of orders'),
    tooltip=['month:T', 'orders']
).properties(width=650, height=260, title='Monthly order volume')
```

```
Out[ ]:
```

```
In [ ]: feature_1_insights = """
`order_date` is the most important column for the repeat-purchase model.

Format and quality:
- Stored as a string (e.g. '2011-05-30 22:00:00') and must be converted
  with `pd.to_datetime` before use. The prep notebook does this in B.1.
- 0 unparseable dates in the supplied data. No rows lost to date conversion.

Temporal coverage:
- Roughly 3+ years of order history (typical AdventureWorks span:
  mid-2011 to mid-2014). This is sufficient for one cutoff with a 90-day
  prediction window, and would support multiple time-block validation
  cutoffs if a more rigorous evaluation were required.
- Monthly order volume shows a clear growth trend and seasonality.
Relevant context for the model:
  * The positive-class rate (proportion of customers who reorder in any
    90-day window) is itself time-varying. A cutoff late in the data
    captures a more mature, higher activity customer base than an early
    cutoff would.
  * Customers acquired close to the cutoff have less feature history
    than long tenured customers; the prep notebook's `tenure_days`
    feature is the model's main lever to handle this.

Implications:
- Cutoff choice = max(order_date) - 90 days. This is documented in the
  prep notebook and selected on the basis of the date range observed here.
- The label is built from a single fixed 90-day post-cutoff window. A
  more advanced extension would use rolling cutoffs to estimate temporal
  stability of the model.
"""
```

```
In [ ]: # Do not modify this code
print_tile(size="h3", key='feature_1_insights', value=feature_1_insights)
```

feature_1_insights

`order\_date` is the most important column for the repeat-purchase model. Format and quality: - Stored as a string (e.g. '2011-05-30 22:00:00') and must be converted with `pd.to\_datetime` before use. The prep notebook does this in B.1. - 0 unparseable dates in the supplied data. No rows lost to date conversion. Temporal coverage: - Roughly 3+ years of order history (typical AdventureWorks span: mid-2011 to mid-2014). This is sufficient for one cutoff with a 90-day prediction window, and would support multiple time-block validation cutoffs if a more rigorous evaluation were required. - Monthly order volume shows a clear growth trend and seasonality. Relevant context for the model: * The positive-class rate (proportion of customers who reorder in any 90-day window) is itself time-varying. A cutoff late in the data captures a more mature, higher activity customer base than an early cutoff would. * Customers acquired close to the cutoff have less feature history than long tenured customers; the prep notebook's `tenure\_days` feature is the model's main lever to handle this. Implications: - Cutoff choice = max(order_date) - 90 days. This is documented in the prep notebook and selected on the basis of the date range observed here. - The label is built from a single fixed 90-day post-cutoff window. A more advanced extension would use rolling cutoffs to estimate temporal stability of the model.

B.3 Explore Feature of Interest `sub_total` (and `total_due`)

```
In [ ]: # Money columns drive the Monetary part of RFM. Inspect distribution shape,
# negatives, extreme values, and the relationship between sub_total and total_due.
print('sub_total summary statistics:')
print(df['sub_total'].describe(percentiles=[0.01, 0.25, 0.5, 0.75, 0.95, 0.99, 0.995]).round(2))

print(f'\nNegative sub_total rows : {(df["sub_total"] < 0).sum()}')
print(f'Zero sub_total rows      : {(df["sub_total"] == 0).sum()}')
print(f'Above 99.5th percentile : {(df["sub_total"] > df["sub_total"].quantile(0.995)).sum()} '
      f' (threshold ≈ {df["sub_total"].quantile(0.995):.2f})')

# Relationship sub_total vs total_due (should be ~ sub_total * 1.1 once tax & freight added)
```

```
ratio = df['total_due'] / df['sub_total'].replace(0, np.nan)
print(f'\nratio total_due / sub_total - median: {ratio.median():.4f}, '
      f'p1: {ratio.quantile(0.01):.4f}, p99: {ratio.quantile(0.99):.4f}')
```

sub_total summary statistics:

```
count    31465.00
mean      3491.07
std       11093.45
min         1.37
1%         4.99
25%        56.97
50%       782.99
75%      2366.96
95%     18060.73
99%     61528.83
99.5%    79576.54
max     163930.39
```

Name: sub_total, dtype: float64

Negative sub_total rows : 0

Zero sub_total rows : 0

Above 99.5th percentile : 158 (threshold ≈ 79,576.54)

ratio total_due / sub_total - median: 1.1050, p1: 1.1050, p99: 1.1295

```
In [ ]: # Histogram of sub_total on log scale (the raw scale is dominated by extreme
# values). Log-transformation reveals the underlying bimodality between small online
# orders and large business orders.
money_df = df[['sub_total', 'online_order_flag']].copy()
money_df = money_df[money_df['sub_total'] > 0]
money_df['log10_sub_total'] = np.log10(money_df['sub_total'])
money_df['channel'] = np.where(money_df['online_order_flag'] == 1, 'online', 'offline')

alt.Chart(money_df.sample(n=min(20000, len(money_df)), random_state=0)).mark_bar(opacity=0.6).enc
x=alt.X('log10_sub_total:Q', bin=alt.Bin(maxbins=50),
        title='log10(sub_total in USD)'),
y=alt.Y('count():Q', stack=None, title='Number of orders'),
color=alt.Color('channel:N', title='Channel'),
tooltip=[alt.Tooltip('count():Q', title='count'), 'channel']
).properties(width=600, height=260,
             title='Order amount distribution (log scale) by channel')
```

Out[]:

```
In [ ]: feature_2_insights = """
'sub_total' is the primary money field (excludes tax and freight) and feeds
the Monetary part of the RFM features in the prep notebook.
```

Distribution:

- Heavy right tail typical of retail: median is in the low triple digits while the 99.5th-percentile is in the tens of thousands. A handful of business orders are 1-2 orders of magnitude larger than typical consumer orders.
- No negative values in the supplied data (refunds are not present in sales_order_header). The prep notebook still guards against negatives as a defensive measure.
- Log10(sub_total) shows clear bimodality split by channel:
 - * Online orders cluster around ~\$50-\$500.
 - * Offline orders cluster around ~\$5,000-\$50,000. These are the business / reseller orders.

The model will need to learn that 'large sub_total' is normal for some customers and unusual for others exactly the kind of signal that benefits from the 'online_order_ratio' feature engineered in prep.

Relationship to total_due:

- total_due = sub_total + tax_amount + freight, so total_due / sub_total is ~1.1 (≈10% combined tax + freight) for most orders. Either column is acceptable as the Monetary aggregator; the prep notebook uses sub_total as the primary because it isolates revenue from tax/freight.

Implications for modelling:

- The right tail dominates monetary aggregates (sum, mean) and risks crushing the discrimination between average customers. The prep notebook winsorises sub_total at the 99.5th percentile (cap, not drop).
- Log-transformation is an alternative; left to the modelling notebook to consider if linear models are used.

"""

```
In [ ]: # Do not modify this code
print_tile(size="h3", key='feature_2_insights', value=feature_2_insights)
```

feature_2_insights

`sub\_total` is the primary money field (excludes tax and freight) and feeds the Monetary part of the RFM features in the prep notebook. Distribution: - Heavy right tail typical of retail: median is in the low triple digits while the 99.5th-percentile is in the tens of thousands. A handful of business orders are 1-2 orders of magnitude larger than typical consumer orders. - No negative values in the supplied data (refunds are not present in sales_order_header). The prep notebook still guards against negatives as a defensive measure. - Log10(sub_total) shows clear bimodality split by channel: * Online orders cluster around ~\$50-\$500. * Offline orders cluster around ~\$5,000-\$50,000. These are the business / reseller orders. The model will need to learn that 'large sub_total' is normal for some customers and unusual for others exactly the kind of signal that benefits from the `online\_order\_ratio` feature engineered in prep. Relationship to total due: - total due = sub_total + tax amount + freight, so total due / sub_total is ~1.1 (~10% combined tax + freight) for most orders. Either column is acceptable as the Monetary aggregator; the prep notebook uses sub_total as the primary because it isolates revenue from tax/freight. Implications for modelling: - The right tail dominates monetary aggregates (sum, mean) and risks crushing the discrimination between average customers. The prep notebook winsorises sub_total at the 99.5th percentile (cap, not drop). - Log-transformation is an alternative; left to the modelling notebook to consider if linear models are used.

B.4 Explore Feature of Interest `online_order_flag`

```
In [ ]: # online_order_flag is the channel indicator. Distribution, and shift over time.
print('Channel distribution (1 = online, 0 = offline):')
print(df['online_order_flag'].value_counts(normalize=True).round(4))

# Channel mix by year
df_tmp = df.copy()
df_tmp['order_date_dt'] = pd.to_datetime(df_tmp['order_date'], errors='coerce')
df_tmp['year'] = df_tmp['order_date_dt'].dt.year
channel_by_year = (df_tmp.groupby('year')['online_order_flag']
                  .agg(['count', 'mean'])
                  .rename(columns={'count': 'orders', 'mean': 'pct_online'}))
channel_by_year['pct_offline'] = 1 - channel_by_year['pct_online']
display(channel_by_year.style.format({'pct_online': '{:.2%}', 'pct_offline': '{:.2%}'))
```

Channel distribution (1 = online, 0 = offline):

```
online_order_flag
1.0    0.879
0.0    0.121
Name: proportion, dtype: float64
```

	orders	pct_online	pct_offline
year			
2011	1692	71.34%	28.66%
2012	3840	71.54%	28.46%
2013	14234	88.77%	11.23%
2014	11699	94.61%	5.39%

```
In [ ]: # Stacked-bar chart of online vs offline orders per year.
plot_df = (channel_by_year[['pct_online', 'pct_offline']]
          .rename(columns={'pct_online': 'online', 'pct_offline': 'offline'})
          .reset_index()
          .melt(id_vars='year', var_name='channel', value_name='share'))

alt.Chart(plot_df).mark_bar().encode(
    x=alt.X('year:O', title='Year'),
    y=alt.Y('share:Q', stack='normalize', title='Share of orders',
```



```

        axis=alt.Axis(format='%'),
        color=alt.Color('channel:N', title='Channel'),
        tooltip=['year', 'channel', alt.Tooltip('share:Q', format='.2%')]
    ).properties(width=480, height=260,
        title='Channel mix by year')

```

Out[]:

```

In [ ]: feature_n_insights = """
`online_order_flag` is a binary channel indicator and feeds the
`online_order_ratio` feature engineered in the prep notebook.

Distribution:
- The supplied data is dominated by the online channel: 87.9% of orders
  are online (flag=1) and 12.1% are offline / sales rep (flag=0).
- This 88/12 split matches the missingness pattern of sales_person_id
  (which is NULL on 87.9% of rows) confirming the two columns carry
  the same information and that the prep notebook is correct to drop
  sales_person_id in favour of online_order_flag.
- The mix shifts over time: in the earlier years of the data, offline
  sales (through resellers / sales reps) represents a much larger share;
  by the final year, online dominates. The year-by-year breakdown chart
  above visualises this transition.

Why this matters for repeat-purchase prediction:
- Customers who order online typically have a different repeat-purchase
  pattern (smaller value, more frequent) than customers who order
  through a sales rep (larger-value, less frequent but longer term).
  A customer's pre-cutoff `online_order_ratio` is therefore a leading
  indicator of the cadence and amount of future activity.
- The 12% offline minority is also where the very large business orders
  sit (confirmed in B.3 by the bimodal log-money distribution). These
  customers may have a very different positive-class rate from online
  customers.
- The temporal drift means that a model trained on early years would
  not generalise cleanly to late years. The prep notebook's cutoff
  near the end of the data is the right choice; rolling-cutoff
  evaluation is documented as a future work option.

Caveat:
- Channel and sales_person_id are redundant in this dataset; the model
  drops sales_person_id and keeps the cleaner derived ratio instead.
"""

```

```

In [ ]: # Do not modify this code
print_tile(size="h3", key='feature_n_insights', value=feature_n_insights)

```

`online\_order\_flag` is a binary channel indicator and feeds the `online\_order\_ratio` feature engineered in the prep notebook. Distribution:

- The supplied data is dominated by the online channel: 87.9% of orders are online (flag=1) and 12.1% are offline / sales rep (flag=0). - This 88/12 split matches the missingness pattern of sales_person_id (which is NULL on 87.9% of rows) confirming the two columns carry the same information and that the prep notebook is correct to drop sales_person_id in favour of online_order_flag. - The mix shifts over time: in the earlier years of the data, offline sales (through resellers / sales reps) represents a much larger share; by the final year, online dominates. The year-by-year breakdown chart above visualises this transition. Why this matters for repeat-purchase prediction: - Customers who order online typically have a different repeat-purchase pattern (smaller value, more frequent) than customers who order through a sales rep (larger-value, less frequent but longer term). A customer's pre-cutoff `online\_order\_ratio` is therefore a leading indicator of the cadence and amount of future activity. - The 12% offline minority is also where the very large business orders sit (confirmed in B.3 by the bimodal log-money distribution). These customers may have a very different positive-class rate from online customers. - The temporal drift means that a model trained on early years would not generalise cleanly to late years. The prep notebook's cutoff near the end of the data is the right choice; rolling-cutoff evaluation is documented as a future work option. Caveat: - Channel and sales_person_id are redundant in this dataset; the model drops sales_person_id and keeps the cleaner derived ratio instead.

B.5 Ethics — channel drift and fairness

```
In [4]: !pip install -q utstd
from utstd.ipynrenders import print_tile

ethics_fairness_insights = """
Ethics – channel mix, temporal drift and fairness implications

The 88% online / 12% offline split is not just a modelling concern, it
has fairness implications:

- Customers with low digital engagement (older customers, customers in
  regional or remote areas with poor connectivity, customers in
  communities with low online retail penetration) are under-represented
  in the online channel. A model trained on overall data will be
  LESS ACCURATE on these segments, and any retention-marketing campaign
  driven by that model will systematically under-serve them.
- The channel shift over time (offline → online) means the model is
  fitted on a customer base whose composition has changed. The model
  may therefore generalise poorly to the most recent acquisitions if
  they differ in channel preference from earlier cohorts.

Engagement feedback loop:
- Customers contacted by previous campaigns are more likely to order in
  the next window. This inflates their feature values, raises their
  future scores, and increases their probability of being contacted
  again. Left unchecked, this self-reinforcing loop entrenches existing
  inequities between engaged and disengaged customers.

Mitigations recommended for production:
- Monitor per-segment recall (online vs offline; per territory) and
  trigger remediation if the gap exceeds an agreed threshold.
- Hold out a randomised slice of every campaign so retraining sees
  representative behaviour, not just engagement-amplified behaviour.
"""
print_tile(size="h3", key='ethics_fairness_insights', value=ethics_fairness_insights)
```

```
12.9/12.9 MB 71.7 MB/s eta 0:00:00
4.9/4.9 MB 72.6 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are i
nstalled. This behaviour is the source of the following dependency conflicts.
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatib
le.
hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
ethics_fairness_insights
```

Ethics — channel mix, temporal drift and fairness implications The 88% online / 12% offline split is not just a modelling concern, it has fairness implications: - Customers with low digital engagement (older customers, customers in regional or remote areas with poor connectivity, customers in communities with low online retail penetration) are under-represented in the online channel. A model trained on overall data will be LESS ACCURATE on these segments, and any retention-marketing campaign driven by that model will systematically under-serve them. - The channel shift over time (offline → online) means the model is fitted on a customer base whose composition has changed. The model may therefore generalise poorly to the most recent acquisitions if they differ in channel preference from earlier cohorts. Engagement feedback loop: - Customers contacted by previous campaigns are more likely to order in the next window. This inflates their feature values, raises their future scores, and increases their probability of being contacted again. Left unchecked, this self-reinforcing loop entrenches existing inequities between engaged and disengaged customers. Mitigations recommended for production: - Monitor per-segment recall (online vs offline; per territory) and trigger remediation if the gap exceeds an agreed threshold. - Hold out a randomised slice of every campaign so retraining sees representative behaviour, not just engagement-amplified behaviour.